

LAB EXPERIMENT – 4

Building and Automating Machine Learning Models Using AutoML with Vertex AI

Course Outcome: CO1

Aim

To build and automate a machine learning model development workflow using AutoML techniques by automatically selecting and optimizing machine learning algorithms, evaluating model performance, and tracking experiments using MLflow for effective MLOps implementation.

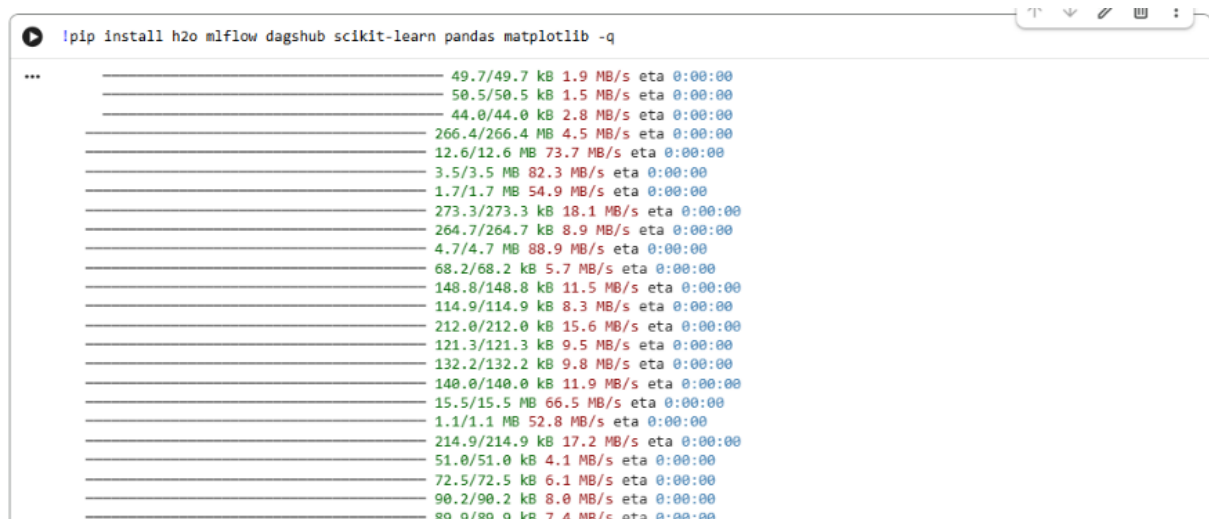
Steps

1. Implement an automated machine learning pipeline using **H2O AutoML** in a Google Colab environment.
2. Automate model selection, hyperparameter tuning, and performance evaluation without manual algorithm selection.
3. Compare multiple machine learning models and identify the best-performing model using AutoML leaderboard ranking.
4. Integrate **MLflow** for tracking experiments, parameters, metrics, and model artifacts.
5. Demonstrate an end-to-end MLOps workflow including model training, evaluation, versioning, and reproducibility.

Step 1: Set up Google Colab Environment

- Create a new Google Colab notebook.
- Install the required libraries for AutoML and MLOps.

`!pip install h2o mlflow dagshub scikit-learn pandas matplotlib -q`



```
!pip install h2o mlflow dagshub scikit-learn pandas matplotlib -q
...
49.7/49.7 kB 1.9 MB/s eta 0:00:00
50.5/50.5 kB 1.5 MB/s eta 0:00:00
44.0/44.0 kB 2.8 MB/s eta 0:00:00
266.4/266.4 MB 4.5 MB/s eta 0:00:00
12.6/12.6 MB 73.7 MB/s eta 0:00:00
3.5/3.5 MB 82.3 MB/s eta 0:00:00
1.7/1.7 MB 54.9 MB/s eta 0:00:00
273.3/273.3 kB 18.1 MB/s eta 0:00:00
264.7/264.7 kB 8.9 MB/s eta 0:00:00
4.7/4.7 MB 88.9 MB/s eta 0:00:00
68.2/68.2 kB 5.7 MB/s eta 0:00:00
148.8/148.8 kB 11.5 MB/s eta 0:00:00
114.9/114.9 kB 8.3 MB/s eta 0:00:00
212.0/212.0 kB 15.6 MB/s eta 0:00:00
121.3/121.3 kB 9.5 MB/s eta 0:00:00
132.2/132.2 kB 9.8 MB/s eta 0:00:00
140.0/140.0 kB 11.9 MB/s eta 0:00:00
15.5/15.5 MB 66.5 MB/s eta 0:00:00
1.1/1.1 MB 52.8 MB/s eta 0:00:00
214.9/214.9 kB 17.2 MB/s eta 0:00:00
51.0/51.0 kB 4.1 MB/s eta 0:00:00
72.5/72.5 kB 6.1 MB/s eta 0:00:00
90.2/90.2 kB 8.0 MB/s eta 0:00:00
89.9/89.9 kB 7.4 MB/s eta 0:00:00
```

Step 2: Import Required Libraries

```
import h2o
```

```
from h2o.automl import H2OAutoML
```

```
import pandas as pd
```

```
import mlflow
```

```
import dagshub
```

```
[2] import h2o
✓ 5s from h2o.automl import H2OAutoML
import pandas as pd
import mlflow
import dagshub
```

Step 3: Initialize H2O AutoML Environment

Start the H2O server in Colab.

```
h2o.init()
```

```
[4] h2o.init()
✓ 10s ... Checking whether there is an H2O instance running at http://localhost:54321..... not found.
Attempting to start a local H2O server...
Java Version: openjdk version "17.0.19" 2026-04-21; OpenJDK Runtime Environment (build 17.0.19+10-1-22.04.2-Ubuntu); OpenJDK 64-Bit S
Starting server from /usr/local/lib/python3.12/dist-packages/h2o/backend/bin/h2o.jar
Ice root: /tmp/tmp5du2e1zi
JVM stdout: /tmp/tmp5du2e1zi/h2o_unknownUser_started_from_python.out
JVM stderr: /tmp/tmp5du2e1zi/h2o_unknownUser_started_from_python.err
Server is running at http://127.0.0.1:54321
Connecting to H2O server at http://127.0.0.1:54321 ... successful.

H2O_cluster_uptime: 03 secs
H2O_cluster_timezone: Etc/UTC
H2O_data_parsing_timezone: UTC
H2O_cluster_version: 3.46.0.11
H2O_cluster_version_age: 2 months and 8 days
H2O_cluster_name: H2O_from_python_unknownUser_owwje5
H2O_cluster_total_nodes: 1
H2O_cluster_free_memory: 3.168 Gb
H2O_cluster_total_cores: 2
H2O_cluster_allowed_cores: 2
H2O_cluster_status: locked, healthy
H2O_connection_url: http://127.0.0.1:54321
H2O_connection_proxy: {"http": null, "https": null, "colab_language_server": "/usr/colab/bin/language_service"}
H2O_internal_security: False
Python_version: 3.12.13 final
```

Step 4: Prepare Dataset

- Create or upload a dataset.
- Perform basic preprocessing.
- Convert the dataset into H2O data format.

```
import pandas as pd
```

```
data = {
    "study_hours": [1,2,3,4,5,6,7,8],
    "attendance": [50,55,60,70,75,85,90,95],
    "internal_marks": [35,40,45,60,70,75,85,90],
    "result": ["Fail","Fail","Fail","Pass","Pass","Pass","Pass","Pass"]
}
```

```
df = pd.DataFrame(data)

df.to_csv("dataset.csv", index=False)

print("Dataset created successfully")

df
```

```
import pandas as pd

data = {
    "study_hours": [1,2,3,4,5,6,7,8],
    "attendance": [50,55,60,70,75,85,90,95],
    "internal_marks": [35,40,45,60,70,75,85,90],
    "result": ["Fail","Fail","Fail","Pass","Pass","Pass","Pass","Pass"]
}

df = pd.DataFrame(data)

df.to_csv("dataset.csv", index=False)

print("Dataset created successfully")

df
```

Dataset created successfully

	study_hours	attendance	internal_marks	result
0	1	50	35	Fail
1	2	55	40	Fail
2	3	60	45	Fail
3	4	70	60	Pass
4	5	75	70	Pass
5	6	85	75	Pass
6	7	90	85	Pass
7	8	95	90	Pass

Next steps: [Generate code with df](#) [New interactive sheet](#)

Then

```
data = pd.read_csv("dataset.csv")

h2o_data = h2o.H2OFrame(data)

h2o_data
```

```
[6]
✓ 0s data = pd.read_csv("dataset.csv")

h2o_data = h2o.H2OFrame(data)

h2o_data
```

Parse progress: | (done) 100%

	study_hours	attendance	internal_marks	result
1	50	35	Fail	
2	55	40	Fail	
3	60	45	Fail	
4	70	60	Pass	
5	75	70	Pass	
6	85	75	Pass	
7	90	85	Pass	
8	95	90	Pass	

[8 rows x 4 columns]

Step 5: Define Features and Target Variable

Your prediction target is the result (Pass/Fail).

```
features = [  
    "study_hours",  
    "attendance",  
    "internal_marks"  
]  
  
target = "result"
```

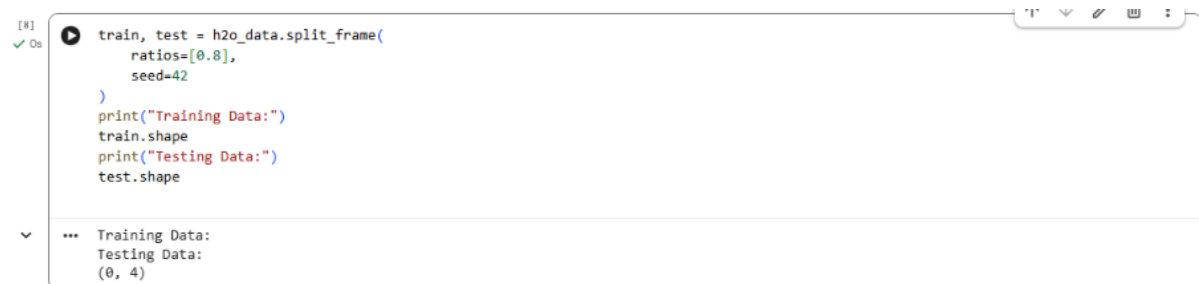


```
[7] ✓ 0s • features = [  
      "study_hours",  
      "attendance",  
      "internal_marks"  
  ]  
  target = "result"
```

Step 6: Split Dataset (Training and Testing)

Convert the data into training and testing sets.

```
train, test = h2o_data.split_frame(  
    ratios=[0.8],  
    seed=42  
)  
  
print("Training Data:")  
  
train.shape  
  
print("Testing Data:")  
  
test.shape
```



```
[8] ✓ 0s • train, test = h2o_data.split_frame(  
      ratios=[0.8],  
      seed=42  
  )  
  print("Training Data:")  
  train.shape  
  print("Testing Data:")  
  test.shape  
  
*** Training Data:  
Testing Data:  
(0, 4)
```

Step 7: Run H2O AutoML Training

AutoML will automatically try different ML algorithms:

- Gradient Boosting Machine (GBM)
- Random Forest
- XGBoost

- Neural Network
- Stacked Ensemble

[illegible]

Step 8: View AutoML Leaderboard

This shows all automatically generated models.

Run:

```
leaderboard = automl.leaderboard
```

leaderboard

```
[10] ✓ Os leaderboard = automl.leaderboard
leaderboard
```

	model_id	auc	logloss	aucpr	mean_per_class_error	rmse	mse
	XRT_1_AutoML_1_20260731_53738	1	0.0525998	1	0	0.0969211	0.00939371
	GBM_5_AutoML_1_20260731_53738	1	0	1	0	0	0
	GLM_1_AutoML_1_20260731_53738	1	0.106802	1	0	0.169975	0.0288914
	DRF_1_AutoML_1_20260731_53738	1	0.0557859	1	0	0.127279	0.0162
	GBM_grid_1_AutoML_1_20260731_53738_model_1	1	0.0186413	1	0	0.0488917	0.0023904
	DeepLearning_1_AutoML_1_20260731_53738	0.866667	2.44847	0.93834	0.1	0.611423	0.373837
	XGBoost_1_AutoML_1_20260731_53738	0.5	0.693147	0.625	0.5	0.5	0.25
	XGBoost_3_AutoML_1_20260731_53738	0.5	0.693147	0.625	0.5	0.5	0.25
	XGBoost_2_AutoML_1_20260731_53738	0.5	0.693147	0.625	0.5	0.5	0.25
	XGBoost_grid_1_AutoML_1_20260731_53738_model_1	0.3	0.703898	0.527391	0.5	0.504221	0.254239

[10 rows x 7 columns]

Step 9: Select Best Model

```
best_model = automl.leader
```

best_model

```
[11] ✓ 0s ▶ best_model = automl.leader
best_model

...
Model Details
=====
H2ORandomForestEstimator : Distributed Random Forest
Model Key: XRT_1_AutoML_1_20260731_53738

Model Summary:
number_of_trees number_of_internal_trees model_size_in_bytes min_depth max_depth mean_depth min_leaves max
41.0 41.0 3458.0 1.0 2.0 1.0487804 2.0

ModelMetricsBinomial: drf
** Reported on train data. **

MSE: 0.0068633572505695855
RMSE: 0.08284538158865336
LogLoss: 0.03935377554634912
Mean Per-Class Error: 0.0
AUC: 1.0
AUCPR: 1.0
Gini: 1.0
```

Step 10: Evaluate Best Model

Before that check

train.shape

test.shape

```
train.shape
(8, 4)

test.shape
... (0, 4)
```

Recreate dataset split with a larger test portion

```
import pandas as pd
```

```
import random
```

```
data = []
```

```
for i in range(100):
```

```
    study_hours = random.randint(1, 10)
```

```
    attendance = random.randint(50, 100)
```

```
    internal_marks = random.randint(35, 100)
```

```
    if (study_hours >= 5 and attendance >= 75 and internal_marks >= 60):
```

```

        result = "Pass"
    else:
        result = "Fail"
    data.append([
        study_hours,
        attendance,
        internal_marks,
        result
    ])
df = pd.DataFrame(
    data,
    columns=[
        "study_hours",
        "attendance",
        "internal_marks",
        "result"
    ]
)
df.to_csv("dataset.csv", index=False)
df.head()

```

```

...
   study_hours  attendance  internal_marks  result
0           4           95             97     Fail
1           3           86            100     Fail
2           1          100             49     Fail
3           2           98             51     Fail
4           5           72             59     Fail

```

Next steps:

[Generate code with df](#)
[New interactive sheet](#)

Convert to H2OFrame

```
data = pd.read_csv("dataset.csv")
```

```
h2o_data = h2o.H2OFrame(data)

h2o_data["result"] = h2o_data["result"].asfactor()

h2o_data.head()
```

```
data = pd.read_csv("dataset.csv")

h2o_data = h2o.H2OFrame(data)

h2o_data["result"] = h2o_data["result"].asfactor()

h2o_data.head()
```

Parse progress: (done) 100%

study_hours	attendance	internal_marks	result
4	95	97	Fail
3	86	100	Fail
1	100	49	Fail
2	98	51	Fail
5	72	59	Fail
6	56	46	Fail
2	69	98	Fail
7	98	93	Pass
3	59	63	Fail
9	66	98	Fail

[10 rows x 4 columns]

Split the dataset

```
train, test = h2o_data.split_frame(
    ratios=[0.8],
    seed=42
)

print("Train:", train.shape)

print("Test:", test.shape)
```

```
train, test = h2o_data.split_frame(
    ratios=[0.8],
    seed=42
)

print("Train:", train.shape)
print("Test:", test.shape)
```

Train: (82, 4)
Test: (18, 4)

Retrain AutoML

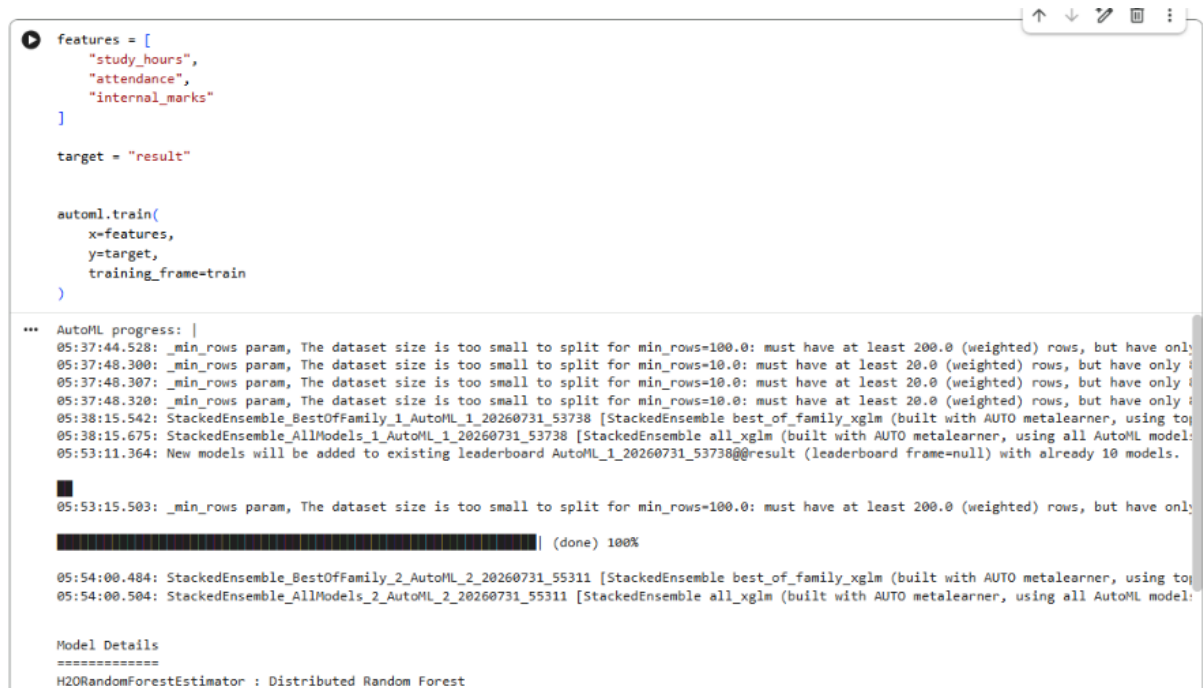
```

features = [
    "study_hours",
    "attendance",
    "internal_marks"
]

target = "result"

automl.train(
    x=features,
    y=target,
    training_frame=train
)

```



```

features = [
    "study_hours",
    "attendance",
    "internal_marks"
]

target = "result"

automl.train(
    x=features,
    y=target,
    training_frame=train
)

```

AutoML progress: |

```

05:37:44.528: _min_rows param, The dataset size is too small to split for min_rows=100.0: must have at least 200.0 (weighted) rows, but have only
05:37:48.300: _min_rows param, The dataset size is too small to split for min_rows=10.0: must have at least 20.0 (weighted) rows, but have only
05:37:48.307: _min_rows param, The dataset size is too small to split for min_rows=10.0: must have at least 20.0 (weighted) rows, but have only
05:37:48.320: _min_rows param, The dataset size is too small to split for min_rows=10.0: must have at least 20.0 (weighted) rows, but have only
05:38:15.542: StackedEnsemble_BestOfFamily_1_AutoML_1_20260731_53738 [StackedEnsemble best_of_family_xglm (built with AUTO metalearner, using top
05:38:15.675: StackedEnsemble_AllModels_1_AutoML_1_20260731_53738 [StackedEnsemble all_xglm (built with AUTO metalearner, using all AutoML model
05:53:11.364: New models will be added to existing leaderboard AutoML_1_20260731_53738@result (leaderboard frame=null) with already 10 models.

```

05:53:15.503: _min_rows param, The dataset size is too small to split for min_rows=100.0: must have at least 200.0 (weighted) rows, but have only

(done) 100%

```

05:54:00.484: StackedEnsemble_BestOfFamily_2_AutoML_2_20260731_55311 [StackedEnsemble best_of_family_xglm (built with AUTO metalearner, using top
05:54:00.504: StackedEnsemble_AllModels_2_AutoML_2_20260731_55311 [StackedEnsemble all_xglm (built with AUTO metalearner, using all AutoML model

```

Model Details

```

=====
H2ORandomForestEstimator : Distributed Random Forest

```

Evaluate

```

best_model = automl.leader

performance = best_model.model_performance(test)

performance

```

```
best_model = automl.leader
performance = best_model.model_performance(test)
performance
```

...

ModelMetricsBinomial: drf
** Reported on test data. **

MSE: 0.35796318329036947
RMSE: 0.5983002451030498
LogLoss: 4.481286038227857
Mean Per-Class Error: 0.06666666666666667
AUC: 0.9333333333333333
AUCPR: 0.6000000000000001
Gini: 0.8666666666666667

Confusion Matrix (Act/Pred) for max f1 @ threshold = 1.0

	Fail	Pass	Error	Rate
Fail	13.0	2.0	0.1333	(2.0/15.0)
Pass	0.0	3.0	0.0	(0.0/3.0)
Total	13.0	5.0	0.1111	(2.0/18.0)

Maximum Metrics: Maximum metrics at their respective thresholds

	metric	threshold	value	idx
	max f1	1.0	0.7500000	0.0
	max f2	1.0	0.8823529	0.0
	max f0point5	1.0	0.6521739	0.0
	max accuracy	1.0	0.8888889	0.0
	max precision	1.0	0.6	0.0
	max recall	1.0	1.0	0.0

MLOps part of the skill experiment: experiment tracking, model saving, and repository integration

Step 1: Extract Evaluation Metrics

Run:

```
accuracy = performance.accuracy()[0][1]
```

```
precision = performance.precision()[0][1]
```

```
recall = performance.recall()[0][1]
```

```
f1_score = performance.F1()[0][1]
```

```
print("Accuracy :", accuracy)
```

```
print("Precision:", precision)
```

```
print("Recall :", recall)
```

```
print("F1 Score :", f1_score)
```

Keep these values for your report.

```
[25]
✓ Os
▶ accuracy = performance.accuracy()[0][1]
  precision = performance.precision()[0][1]
  recall = performance.recall()[0][1]
  f1_score = performance.f1()[0][1]
  print("Accuracy :", accuracy)
  print("Precision:", precision)
  print("Recall   :", recall)
  print("F1 Score :", f1_score)

▼ ... Accuracy : 0.8888888888888888
  Precision: 0.6
  Recall   : 1.0
  F1 Score : 0.7499999999999999
```

Step 2: Setup MLflow Tracking with DagsHub

We have a repository:

<https://dagshub.com/sirisha/mlops-shared-repo>

Run:

```
import dagshub

import mlflow

dagshub.init(
    repo_owner="sirisha",
    repo_name="mlops-shared-repo",
    mlflow=True
)

mlflow.set_experiment("H2O_AutoML_Experiment")
```

```
▶ import dagshub
  import mlflow

  dagshub.init(
    repo_owner="sirisha",
    repo_name="mlops-shared-repo",
    mlflow=True
  )

  mlflow.set_experiment("H2O_AutoML_Experiment")

... !!! AUTHORIZATION REQUIRED !!!

Open the following link in your browser to authorize the client:
https://dagshub.com/login/oauth/authorize?state=4aad02e5-90c1-42d5-ad2b-7393a599462a&client\_id=32b60ba385aa7cecf24046c

Accessing as sirisha
Initialized MLflow to track repo "sirisha/mlops-shared-repo"
Repository sirisha/mlops-shared-repo initialized!
2026/07/31 05:59:35 INFO mlflow.tracking.fluent: Experiment with name 'H2O_AutoML_Experiment' does not exist. Creating
<Experiment: artifact_location='mlflow-artifacts:/bcb860dc9d7142f8a70ebc6e8dec4a45', creation_time=1785477575980,
effective_trace_archival_retention=None, experiment_id='2', last_update_time=1785477575980, lifecycle_stage='active',
name='H2O_AutoML_Experiment', tags={}, trace_location=None, workspace='default'>
```

Step 3: Log AutoML Parameters and Metrics

Run:

```
with mlflow.start_run():
```

```
    # Parameters
```

```
    mlflow.log_param(
```

```
        "algorithm",
```

```
        "H2O AutoML"
```

```
    )
```

```
    mlflow.log_param(
```

```
        "max_models",
```

```
        10
```

```
    )
```

```
    mlflow.log_param(
```

```
        "dataset",
```

```
        "student_performance"
```

```
    )
```

```
    # Metrics
```

```
    mlflow.log_metric(
```

```
        "accuracy",
```

```
        accuracy
```

```
    )
```

```
    mlflow.log_metric(
```

```
        "precision",
```

```
        precision
```

```
    )
```

```
    mlflow.log_metric(
```

```
        "recall",
```

```
        recall
```

```
    )
```

```
    mlflow.log_metric(
```

```
        "f1_score",
```

```

        f1_score
    )
print("MLflow logging completed")

```

```

... View run gaudy-kite-303 at: https://dagshub.com/sirisha/mlops-shared-repo.mlflow/#/experiments/2/runs/c92eced6470d
View experiment at: https://dagshub.com/sirisha/mlops-shared-repo.mlflow/#/experiments/2
MLflow logging completed

```

Step 4: Save the Best AutoML Model

```

model_path = h2o.save_model(
    model=best_model,
    path="/content/automl_model"
)
print(model_path)

```

```

[28] ✓ Os
model_path = h2o.save_model(
    model=best_model,
    path="/content/automl_model"
)

print(model_path)

... /content/automl_model/XRT_1_AutoML_1_20260731_53738

```

Step 5: Log Model Artifact into MLflow

```

with mlflow.start_run():
    mlflow.log_artifact(
        model_path,
        artifact_path="model"
    )
print("Model artifact logged")

```

```

... View run incongruous-tern-262 at: https://dagshub.com/sirisha/mlops-shared-repo.mlflow/#/experiments/2/runs/a877e9
View experiment at: https://dagshub.com/sirisha/mlops-shared-repo.mlflow/#/experiments/2
Model artifact logged

```

Step 5: Test Model Prediction

Create sample input:

```

new_data = h2o.H2OFrame(
    {
        "study_hours":[8],
        "attendance":[95],

```

```

        "internal_marks":[90]
    }
)

```

```

[30] new_data = h2o.H2OFrame(
    {
        "study_hours":[8],
        "attendance":[95],
        "internal_marks":[90]
    }
)

```

Parse progress:  (done) 100%

```
prediction = best_model.predict(new_data)
```

```
prediction
```

```

[31] prediction = best_model.predict(new_data)

prediction

```

drf prediction progress:  (done) 100%

	predict	Fail	Pass
Pass	0	1	

[1 row x 3 columns]

Step 6: Save Experiment Files

Create a result file:

```

results = pd.DataFrame(
    {
        "Model":["H2O AutoML Best Model"],
        "Accuracy":[accuracy],
        "Precision":[precision],
        "Recall":[recall],
        "F1 Score":[f1_score]
    }
)

results.to_csv(
    "automl_results.csv",
    index=False
)

results

```

```

results = pd.DataFrame(
    {
        "Model":["H2O AutoML Best Model"],
        "Accuracy":[accuracy],
        "Precision":[precision],
        "Recall":[recall],
        "F1 Score":[f1_score]
    }
)

results.to_csv(
    "automl_results.csv",
    index=False
)

results

```

	Model	Accuracy	Precision	Recall	F1 Score
0	H2O AutoML Best Model	0.888889	0.6	1.0	0.75

Step 7: Push Files to DagsHub

Check files:

`!ls -la`

```

!ls -la

```

	total	28
drwxr-xr-x	1	root root 4096 Jul 31 06:06 .
drwxr-xr-x	1	root root 4096 Jul 31 05:15 ..
drwxr-xr-x	2	root root 4096 Jul 31 06:02 automl_model
-rw-r--r--	1	root root 109 Jul 31 06:06 automl_results.csv
drwxr-xr-x	4	root root 4096 Jun 4 13:32 .config
-rw-r--r--	1	root root 1355 Jul 31 05:48 dataset.csv
drwxr-xr-x	1	root root 4096 Jun 4 13:32 sample_data

from getpass import getpass

token = getpass("Enter DagsHub Token: ")

Enter your token

Clone using token

`!git clone https://dagshub.com/sirisha/mlops-shared-repo.git`

```

[5] from getpass import getpass
    dagshub_token = getpass("Enter DagsHub Default Access Token: ")

Enter DagsHub Default Access Token: .....

[6] !git clone https://sirisha:$dagshub_token@dagshub.com/sirisha/mlops-shared-repo.git

```

```

*** Cloning into 'mlops-shared-repo'...
remote: Enumerating objects: 4, done.
remote: Counting objects: 100% (4/4), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 4 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (4/4), done.

```

If the token is not accepted, then use this

`!rm -rf /content/mlops-shared-repo`

from getpass import getpass

```
token = getpass("Enter DagsHub Token: ")
```

```
!git clone https://dagshub.com/sirisha/mlops-shared-repo.git
```

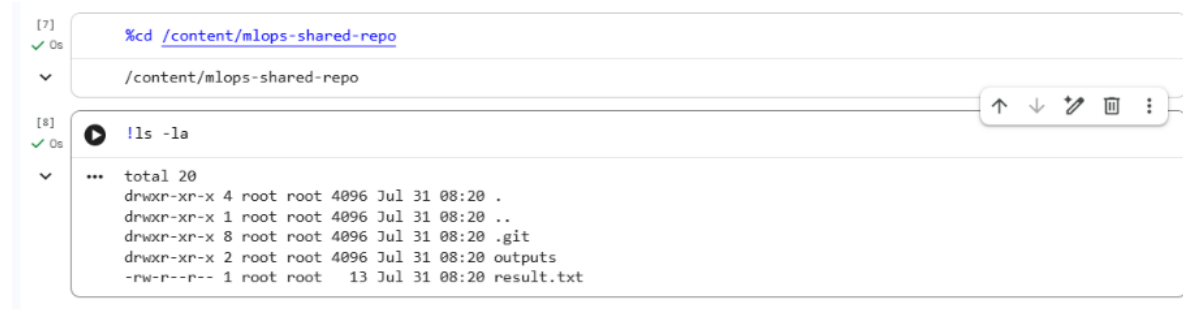
Step 8: Move into the repository

```
%cd /content/mlops-shared-repo
```

```
!ls -la
```

You should see:

```
.git
```



The screenshot shows a Jupyter Notebook terminal with two cells. The first cell contains the command `%cd /content/mlops-shared-repo` and the output is `/content/mlops-shared-repo`. The second cell contains the command `!ls -la` and the output is a directory listing showing the `.git` directory and `outputs` and `result.txt` files.

```
[7] %cd /content/mlops-shared-repo
✓ Os /content/mlops-shared-repo

[8] !ls -la
✓ Os
... total 20
drwxr-xr-x 4 root root 4096 Jul 31 08:20 .
drwxr-xr-x 1 root root 4096 Jul 31 08:20 ..
drwxr-xr-x 8 root root 4096 Jul 31 08:20 .git
drwxr-xr-x 2 root root 4096 Jul 31 08:20 outputs
-rw-r--r-- 1 root root 13 Jul 31 08:20 result.txt
```

Step 9: Create automl_results.csv

Use your obtained metrics:

```
import pandas as pd
```

```
results = pd.DataFrame(
    {
        "Model": ["H2O AutoML Best Model"],
        "Accuracy": [0.888888],
        "Precision": [0.6],
        "Recall": [1.0],
        "F1 Score": [0.75]
    }
)
```

```
results.to_csv(
    "automl_results.csv",
    index=False
)
```

Results

	Model	Accuracy	Precision	Recall	F1 Score	
0	H2O AutoML Best Model	0.888888	0.6	1.0	0.75	 

Step 10: Check files

`!ls -la`

You should see:

`.git`

`dataset.csv`

`automl_results.csv`

`outputs`

`result.txt`

```
[15]  !ls -la
✓ Os
... total 28
drwxr-xr-x 4 root root 4096 Jul 31 08:27 .
drwxr-xr-x 1 root root 4096 Jul 31 08:20 ..
-rw-r--r-- 1 root root 85 Jul 31 08:27 automl_results.csv
-rw-r--r-- 1 root root 149 Jul 31 08:27 dataset.csv
drwxr-xr-x 8 root root 4096 Jul 31 08:20 .git
drwxr-xr-x 2 root root 4096 Jul 31 08:20 outputs
-rw-r--r-- 1 root root 13 Jul 31 08:20 result.txt
```

Step 11: Add files to Git

`!git add .`

Check:

`!git status`

```
[16] !git add .
✓ Os

[17]  !git status
✓ Os
... On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   automl_results.csv
    new file:   dataset.csv
```

Step 12: Commit

`!git config --global user.name "Sirisha-Potluri2011"`

`!git config --global user.email "sirisha@klh.edu.in"`

!git add .

!git commit -m "Completed H2O AutoML MLOps experiment"

```
[19]
✓ 0s !git config --global user.name "Sirisha-Potluri2011"
!git config --global user.email "sirisha@klh.edu.in"

[20]
✓ 0s !git add .
!git commit -m "Completed H2O AutoML MLOps experiment"

... [main 7f04dbb] Completed H2O AutoML MLOps experiment
2 files changed, 11 insertions(+)
create mode 100644 automl_results.csv
create mode 100644 dataset.csv
```

Step 13: Push to DagsHub

!git push origin main

Step 14: Push to DagsHub

!git push origin main

If it asks for authentication:

Username:

sirisha

Password:

Your DagsHub Access Token

```
[21]
✓ 1s !git push origin main

... Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 2 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 570 bytes | 570.00 KiB/s, done.
Total 4 (delta 0), reused 0 (delta 0), pack-reused 0
To https://dagshub.com/sirisha/mlops-shared-repo.git
0eac30c..7f04dbb main -> main
```

After successful push, refresh your DagsHub repository page and verify that dataset.csv and automl_results.csv are visible.

